Diagrama_de_Classes

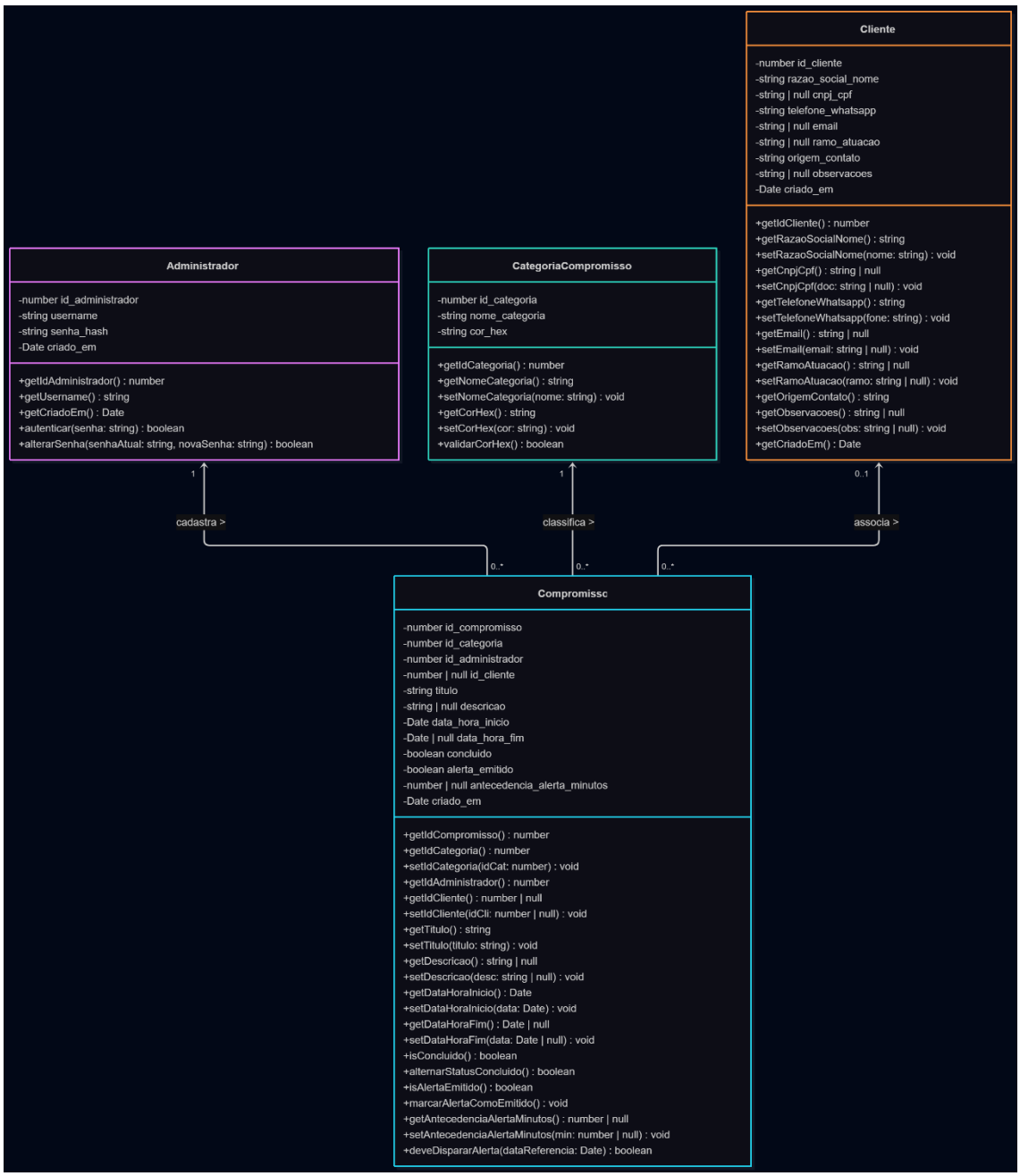
Esse documento apresenta o diagrama de classes da API do projeto.

0. Alterações:

REV00:

Criação do Documento.

1. DIAGRAMA DE CLASSES:



É possível encontrar o código **Mermaid Diagram** para criação desse diagrama no onedrive.

2. DESCRIÇÃO DAS CLASSES DO DIAGRAMA

A seguir, segue o diagrama descrito e detalhado classe por classe:

Classe: Administrador

Finalidade: Representar o gestor/administrador autenticado no sistema. Armazena os dados de credencial e perfil para dar suporte direto ao fluxo de login (CS-003) e restrição de acesso ao painel interno da aplicação (RF-010 / US-10), garantindo a segurança de acesso (RNF-005) e a rastreabilidade das ações executadas na agenda.

Atributos:

Visibilidade	Atributo	Tipo em TypeScript	Propósito / Descrição
- (privado)	id_administrador	number	Identificador único no PostgreSQL (Primary Key / SERIAL).
- (privado)	username	string	Credencial textual de acesso informada no login (VARCHAR(50) / UNIQUE).
- (privado)	senha_hash	string	Hash criptografado da senha (VARCHAR(255)). Nunca exposto fora da classe.
- (privado)	criado_em	Date	Timestamp de criação do registro (TIMESTAMP).

Métodos:

Visibilidade	Método	Retorno (TS)	Propósito / Descrição
+ (público)	getIdAdministrador()	number	Getter: Retorna o ID do administrador.
+ (público)	getUsername()	string	Getter: Retorna o username do administrador.
+ (público)	getCriadoEm()	Date	Getter: Retorna a data de criação do registro.
+ (público)	autenticar(senha: string)	boolean	Compara a senha em texto puro digitada no login com o senha_hash guardado na instância (**CS-003**).
+ (público)	alterarSenha(senhaAtual: string, novaSenha: string)	boolean	Valida a senha atual e atualiza o senha_hash com o novo valor criptografado.

Classe: CategoriaCompromisso

Finalidade: Representar a categorização e classificação dos compromissos no ecossistema do sistema. Encapsula o identificador, a descrição textual do tipo do evento (ex: Pregão Eletrônico, Reunião, Prazo, Rotina) e a sua respectiva cor em hexadecimal para destaque visual nas interfaces web e mobile (RNF-001 / RNF-007), atendendo aos requisitos de Classificação de Tipos de

Compromissos (RF-005 / US-05) e dando suporte aos Casos de Uso CS-004 e CS-006.

Atributos:

Visibilidade	Atributo	Tipo em TypeScript	Propósito / Descrição
- (privado)	id_categoria	number	Identificador único da categoria no PostgreSQL (Primary Key / SERIAL).
- (privado)	nome_categoria	string	Nome descritivo único da categoria (ex: *"Pregão Eletrônico"*, *"Reunião"*, *"Prazo"*) (VARCHAR(50) / UNIQUE).
- (privado)	cor_hex	string	Código da cor em formato hexadecimal de 7 caracteres (ex: '#F26522') para identificação visual (VARCHAR(7)).

Métodos:

Visibilidade	Método	Retorno (TS)	Propósito / Descrição
+ (público)	getIdCategoria()	number	Getter: Retorna o ID da categoria.
+ (público)	getNomeCategoria()	string	Getter: Retorna o nome da categoria.

Visibilidade	Método	Retorno (TS)	Propósito / Descrição
+ (público)	setNomeCategoria(nome: string)	void	Setter: Atualiza o nome da categoria com validação de preenchimento obrigatório.
+ (público)	getCorHex()	string	Getter: Retorna o código hexadecimal da cor.
+ (público)	setCorHex(cor: string)	void	Setter: Atualiza o código hexadecimal da cor garantindo validação prévia.
+ (público)	validarCorHex()	boolean	Método de domínio que valida se a string cor_hex possui a estrutura hexadecimal correta de 7 caracteres (ex: /^#[0-9A-F]{6}$/i).

Classe: Cliente

Finalidade: Representar o cliente ou potencial interessado (lead) captado pelo site ou cadastrado manualmente no sistema. Encapsula as informações de contato, identificação e histórico para atender aos requisitos de Cadastro de Interessados (RF-003 / US-03) e Gerenciamento de Clientes (RF-009 / US-09),

além de dar suporte aos relatórios de captação em Excel (RF-008 / US-08 / CS-008) e à exibição de portfólio de casos de sucesso anonimizados (RF-001 / RNF-002).

Atributos:

Visibilidade	Atributo	Tipo em TypeScript	Propósito / Descrição
- (privado)	id_cliente	number	Identificador único do cliente no PostgreSQL (Primary Key / SERIAL).
- (privado)	razao_social_nome	string	Nome completo ou Razão Social do cliente/empresa (VARCHAR(150)).
- (privado)	cnpj_cpf	string | null	Documento de identificação com validação de duplicidade (VARCHAR(20) / UNIQUE). Não requisitado no primeiro contato (via site), mas passível de preenchimento posterior pelo administrador no painel.
- (privado)	telefone_whatsapp	string	Canal de comunicação direta via WhatsApp (VARCHAR(20)).

Visibilidade	Atributo	Tipo em TypeScript	Propósito / Descrição
- (privado)	email	string | null	Endereço de e-mail corporativo ou pessoal (VARCHAR(100)).
- (privado)	ramo_atuacao	string | null	Ramo de atividade para métricas e exibição de casos anonimizados (VARCHAR(100)).
- (privado)	origem_contato	string	Registro da origem da captação (ex: 'Site', 'WhatsApp', 'Manual') (VARCHAR(50)).
- (privado)	observacoes	string | null	Anotações internas sobre o perfil ou negociação (TEXT).
- (privado)	criado_em	Date	Timestamp de cadastro para filtro temporal em relatórios (TIMESTAMP).

Métodos:

Visibilidade	Método	Retorno (TS)	Propósito / Descrição
+ (público)	getIdCliente()	number	Getter: Retorna o ID do cliente.

Visibilidade	Método	Retorno (TS)	Propósito / Descrição
+ (público)	getRazaoSocialNome()	string	Getter: Retorna o nome ou razão social.
+ (público)	setRazaoSocialNome(nome: string)	void	Setter: Atualiza o nome com validação obrigatoriedade.
+ (público)	getCnpjCpf()	string | null	Getter: Retorna o CNPJ/CPF cadastrado.
+ (público)	setCnpjCpf(doc: string | null)	void	Setter: Atualiza o documento.
+ (público)	getTelefoneWhatsapp()	string	Getter: Retorna o número de WhatsApp.
+ (público)	setTelefoneWhatsapp(fone: string)	void	Setter: Atualiza o telefone com verificação de preenchimento.
+ (público)	getEmail()	string | null	Getter: Retorna o e-mail.
+ (público)	setEmail(email: string | null)	void	Setter: Atualiza o e-mail.
+ (público)	getRamoAtuacao()	string | null	Getter: Retorna o ramo de atuação.

Visibilidade	Método	Retorno (TS)	Propósito / Descrição
+ (público)	setRamoAtuacao(ramo: string | null)	void	Setter: Atualiza o ramo de atuação.
+ (público)	getOrigemContato()	string	Getter: Retorna a origem da captação.
+ (público)	getObservacoes()	string | null	Getter: Retorna as anotações internas.
+ (público)	setObservacoes(obs: string | null)	void	Setter: Atualiza as observações.
+ (público)	getCriadoEm()	Date	Getter: Retorna a data de criação do registro.

Classe: Compromisso

Finalidade: Representar o evento, pregão eletrônico, reunião, prazo ou tarefa cadastrada na agenda da empresa. Encapsula as regras para marcar tarefas como concluídas (checklist), gerenciar o status de disparo de alerta para o worker de notificações (CS-005) e manter os relacionamentos com o administrador criador, a categoria classificadora e o cliente opcionalmente associado.

Atributos:

Visibilidade	Atributo	Tipo em TypeScript	Propósito / Descrição

Privado	id_compromisso	number	Identificador único do compromisso no PostgreSQL (Primary Key / SERIAL).
Privado	id_categoria	number	Chave estrangeira que referencia a categoria/tipo de compromisso (ex.: Pregão Eletrônico, Reunião).
Privado	id_administrador	number	Chave estrangeira para rastrear o administrador criador do evento (RF-010).
Privado	id_cliente	number | null	Chave estrangeira opcional para vincular um cliente/empresa ao compromisso (NULL para tarefas internas).
Privado	titulo	string	Título ou nome curto do compromisso/tarefa (VARCHAR(150)).
Privado	descricao	string | null	Pauta ou detalhamento complementar do evento (TEXT).
Privado	data_hora_inicio	Date	Timestamp de início do compromisso ou prazo final da tarefa (RF-004).
Privado	data_hora_fim	Date | null	Timestamp previsto para o término do compromisso (TIMESTAMP).
Privado	concluido	boolean	Flag de estado para o controle em formato de checklist de tarefas pendentes (RF-007).

Privado	alerta_emitido	boolean	Flag que indica se o alerta já foi disparado pela rotina diária (CS-005). Evita notificações duplicadas.
Privado	antecedencia_alerta_minutos	number | null	Tempo de antecedência em minutos para disparar o alerta (ex.: 1440 para 24h, ou null se o usuário optar por não receber alerta).
Privado	criado_em	Date	Timestamp de criação do registro no banco.

Métodos:

Visibilidade	**Método**	**Retorno (TS)**	**Propósito / Descrição**
+ (público)	getIdCompromisso()	number	Getter: Retorna o ID do compromisso.
+ (público)	getIdCategoria()	number	Getter: Retorna o ID da categoria associada.
+ (público)	setIdCategoria(idCat: number)	void	Setter: Associa a uma nova categoria.
+ (público)	getIdAdministrador()	number	Getter: Retorna o ID do criador.
+ (público)	getIdCliente()	number | null	Getter: Retorna o ID do cliente vinculado (se houver).

Visibilidade	Método	Retorno (TS)	Propósito / Descrição
+ (público)	setIdCliente(idCli: number | null)	void	Setter: Associa ou remove o vínculo com cliente.
+ (público)	getTitulo()	string	Getter: Retorna o título do compromisso.
+ (público)	setTitulo(titulo: string)	void	Setter: Atualiza o título com validação prévia de preenchimento.
+ (público)	getDescricao()	string | null	Getter: Retorna a descrição complementar.
+ (público)	setDescricao(desc: string | null)	void	Setter: Atualiza a descrição complementar.
+ (público)	getDataHoraInicio()	Date	Getter: Retorna a data/hora de início.
+ (público)	setDataHoraInicio(data: Date)	void	Setter: Altera a data/hora de início. **Regra de Negócio:** Se a data for alterada e o evento não estiver concluído, redefine alerta_emitido = false para permitir novo alerta.

Visibilidade	Método	Retorno (TS)	Propósito / Descrição
+ (público)	getDataHoraFim()	Date | null	Getter: Retorna a data/hora de término.
+ (público)	setDataHoraFim(data: Date | null)	void	Setter: Atualiza a data/hora de término.
+ (público)	isConcluido()	boolean	Getter/Check: Retorna true se a tarefa/compromisso estiver finalizada (**RF-007**).
+ (público)	alternarStatusConcluido()	boolean	Método de Negócio: Inverte o booleano concluido (para marcar/desmarcar o item no checklist da interface).
+ (público)	isAlertaEmitido()	boolean	Getter: Retorna se o alerta já foi emitido pelo worker.
+ (público)	marcarAlertaComoEmitido()	void	Método de Negócio: Chamado após a rotina (**CS-005**) enviar a notificação, alterando alerta_emitido = true.
+ (público)	getAntecedenciaAlertaMinutos()	number | null	Getter: Retorna a antecedência em minutos configurada para o alerta.

Visibilidade	Método	Retorno (TS)	Propósito / Descrição
+ (público)	setAntecedenciaAlertaMinutos(min: number | null)	void	Setter: Define o tempo de antecedência. Se for null, ajusta alerta_emitido = true para que a rotina o ignore.
+ (público)	deveDisparar Alerta(dataReferencia: Date)	boolean	Método de Negócio: Avalia se concluido == false, alerta_emitido == false, antecedencia_alerta_minutos != null e se a dataReferencia já atingiu o momento de notificar.

3. RELACIONAMENTOS:

A estrutura relacional da API garante a consistência do negócio (RF-011 / US-11), associando a autenticação dos gestores, o cadastro de clientes, a categorização dos eventos e o gerenciamento da agenda.

[CATEGORIA_COMPROMISSO] (1) <------ (N) [COMPROMISSO]

[ADMINISTRADOR] (1) <------ (N) [COMPROMISSO]

[CLIENTE] (0..1) <------ (N) [COMPROMISSO]

A) Relacionamento: CategoriaCompromisso — Compromisso

- **Cardinalidade:** $1:N$ *(Uma Categoria para Muitos Compromissos)*
- **Tipo de Vínculo:** Obrigatório *(atributo* id_categoria *na classe* Compromisso*)*
- **Regra de Negócio:**

- Uma instância de CategoriaCompromisso (ex: *Pregão Eletrônico*, *Reunião*, *Prazo*, *Rotina*) classifica um ou múltiplos compromissos na agenda (**RF-005 / US-05**).
- Todo Compromisso deve obrigatoriamente estar associado a exatamente 1 CategoriaCompromisso, permitindo a filtragem no painel e a atribuição da cor hexadecimal (cor_hex).

B) Relacionamento: Administrador — Compromisso

- **Cardinalidade:** $1:N$ *(Um Administrador para Muitos Compromissos)*
- **Tipo de Vínculo:** Obrigatório *(atributo* id_administrador *na classe* Compromisso*)*
- **Regra de Negócio:**
  - Um Administrador autenticado no painel (**RF-010 / US-10 / CS-003**) é responsável por cadastrar e gerenciar múltiplos compromissos e tarefas (**RF-004 / US-04**).
  - Cada Compromisso é registrado sob a responsabilidade de exatamente 1 Administrador, garantindo a rastreabilidade das ações internas.

C) Relacionamento: Cliente — Compromisso

- **Cardinalidade:** $0..1:N$ *(Zero ou Um Cliente para Muitos Compromissos)*
- **Tipo de Vínculo:** Opcional *(atributo* id_cliente *aceita* null *na classe* Compromisso*)*
- **Regra de Negócio:**
  - Um Cliente (potencial lead captado pelo site ou cliente ativo cadastrado) pode estar associado a nenhum ou a múltiplos compromissos da agenda (ex: pregões eletrônicos ou reuniões de alinhamento) (**RF-003, RF-009 / CS-002, CS-007**).
  - Um Compromisso pode estar opcionalmente vinculado a um Cliente. Para tarefas internas, rotinas ou prazos administrativos sem cliente direto, o atributo id_cliente permanece como null.

D) Resumo Tático de Integridade das Classes e Banco

Entidade Origem (Pai)	Entidade Destino (Filha)	Atributo de Vínculo	Obrigatoriedade	Regra de Exclusão (FK Behavior)
CategoriaCompromisso	Compromisso	id_categoria	Sim	ON DELETE RESTRICT (impede excluir categorias com compromissos vinculados).
Administrador	Compromisso	id_administrador	Sim	ON DELETE RESTRICT (preserva o histórico de inserção da agenda).
Cliente	Compromisso	id_cliente	Não (null)	ON DELETE SET NULL (ao remover o cliente, mantém a tarefa na agenda desvincul

Entidade Origem (Pai)	Entidade Destino (Filha)	Atributo de Vínculo	Obrigatoriedade	Regra de Exclusão (FK Behavior)
				ando o ID).